

Relatório de Aprendizado - Machine Learning

Tema: Binarização de Imagens Digitais:

Conversão para Tons de Cinza e Limiarização

Ambiente: Google Colab (Python)

Johnnie Richard Pereira

Resumo

A ideia geral deste documento é refletir sobre a construção do projeto de Binarização de Imagens Digitais, implementado em Python sem o uso de funções de alto nível de bibliotecas de visão computacional. Este projeto é parte do bootcamp sobre Machine Learning, da DIO, e tem como objetivo central converter imagens coloridas (espaço RGB) em representações de tons de cinza (0 a 255) e, subsequentemente, em imagens binárias (0 e 255), aplicando três métodos de limiarização distintos. Todo o pipeline é construído a partir de operações matriciais fundamentais com NumPy e Pillow, expondo explicitamente os algoritmos matemáticos subjacentes.

Em resumo, foram implementados três métodos de conversão para tons de cinza, sendo eles: Luminância (BT.601), Luma (BT.709) e Média Aritmética. Além disso, também utilizamos três técnicas de binarização, a saber: Limiar Global Simples, Método de Otsu e Limiarização Adaptativa Local. O código está disponível publicamente no GitHub como material de estudo para disciplinas de Processamento Digital de Imagens.

Sumário

1. Introdução
2. Fundamentação Teórica
 - 2.1 Espaço de Cor RGB
 - 2.2 Conversão para Tons de Cinza
 - 2.3 Binarização por Limiarização
 - 2.4 Método de Otsu
 - 2.5 Limiarização Adaptativa
3. Implementação
 - 3.1 Arquitetura do Pipeline
 - 3.2 Estrutura do Código
 - 3.3 Funções Principais
4. Análise dos Algoritmos
5. Resultados e Discussão
6. Conclusão
7. Referências Bibliográficas

1. Introdução

A binarização de imagens é uma das operações mais fundamentais em Processamento Digital de Imagens (PDI). Trata-se do processo de converter uma imagem colorida ou em tons de cinza para uma representação binária, na qual cada pixel assume exclusivamente um dos dois valores possíveis: 0 (preto) ou 255 (branco). Essa transformação é amplamente utilizada como etapa de pré-processamento em sistemas de visão computacional, reconhecimento óptico de caracteres (OCR), análise de documentos, segmentação de objetos e diagnóstico médico por imagem.

O presente projeto tem por objetivo implementar, do zero e de forma didática, as principais etapas desse pipeline: (i) a conversão de imagens coloridas para o espaço de intensidade de cinza, e (ii) a aplicação de diferentes algoritmos de limiarização para produzir a imagem binária final. A implementação é realizada em Python, utilizando apenas NumPy para operações matriciais e Pillow para entrada/saída de arquivos de imagem, sem recorrer a funções prontas de bibliotecas como OpenCV ou scikit-image.

Este relatório acompanha o código desenvolvido ao longo do curso de Machine Learning da DIO, tendo por base os materiais de apoio disponibilizados. Dessa forma, disponibilizamos no GitHub o código final do projeto, que serve tanto como documentação técnica quanto como material de referência para estudantes da área.

2. Fundamentação Teórica

2.1 Espaço de Cor RGB

Imagens digitais coloridas são tipicamente representadas no espaço de cor RGB (Red, Green, Blue). Cada pixel é descrito por uma tripla (R, G, B) de valores inteiros no intervalo $[0, 255]$, onde cada componente indica a intensidade do respectivo canal de cor primária. Uma imagem de dimensões $W \times H$ pixels¹ possui, portanto, $W \times H \times 3$ valores inteiros armazenados em um array tridimensional.

¹ Em outras palavras, isso significa largura $\times$ altura da imagem em pixels.

2.2 Conversão para Tons de Cinza

A conversão de uma imagem RGB para escala de cinza resulta em um array bidimensional onde cada pixel disposto neste espaço, definidos por $y(i,j)$, são calculados a partir de seus componentes R, G, B originais. Os três métodos implementados são:

2.2.1). Luminância: Padrão ITU-R BT.601

A conversão de uma imagem colorida para tons de cinza baseia-se na forma como o sistema visual humano percebe as diferentes cores. Embora uma imagem digital seja composta pelas três componentes fundamentais, isto é, vermelho (R), verde (G) e azul (B), o olho humano não percebe todas elas com a mesma intensidade. Nossa visão é mais sensível ao verde, moderadamente sensível ao vermelho e menos sensível ao azul.

Por esse motivo, o processo de cálculo da luminância (ou intensidade de brilho da imagem) atribui pesos diferentes para cada canal de cor. O canal verde recebe o maior peso, pois exerce maior influência na percepção visual humana, enquanto o azul recebe o menor peso. Dessa forma, obtém-se uma representação em escala de cinza mais próxima da maneira como percebemos o brilho e contraste no mundo real.

A luminância (Y) da imagem pode ser calculada pela seguinte expressão:

$$Y = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B$$

Nessa equação, temos que:

- (R) → Representa a intensidade do canal vermelho;
- (G) → Representa a intensidade do canal verde;
- (B) → Representa a intensidade do canal azul;
- (Y) → Representa o valor final de brilho da imagem em tons de cinza;

Assim, o valor final não é obtido por uma simples média entre as cores, mas por uma combinação ponderada que busca reproduzir, de maneira mais fiel, a percepção visual humana.

2.2.2). Luma: Padrão ITU-R BT.709

O modelo de luminância definido pelo padrão ITU-R BT.709 foi desenvolvido para sistemas de televisão de alta definição (HDTV) e dispositivos modernos de exibição digital. Nesse método, os coeficientes associados aos canais de cor foram reajustados para refletir

com maior precisão as características dos displays contemporâneos, especialmente em relação à reprodução das cores e da intensidade luminosa.

Assim como em outros modelos de conversão para tons de cinza, o cálculo considera que o sistema visual humano não percebe todas as cores da mesma maneira. O canal verde continua recebendo o maior peso, pois a visão humana é mais sensível às variações dessa faixa do espectro visível. Já os canais vermelho e azul possuem menor influência na percepção do brilho da imagem, especialmente o azul, que recebe o menor peso entre os três.

Nesse padrão, a luminância (Y) é determinada pela seguinte equação:

$$Y = 0,2126 \cdot R + 0,7152 \cdot G + 0,0722 \cdot B$$

Neste caso, temos que:

- (R) → Representa a intensidade do canal vermelho;
- (G) → Representa a intensidade do canal verde;
- (B) → Representa a intensidade do canal azul;
- (Y) → Corresponde ao valor final de luminância da imagem;

A principal diferença em relação ao modelo anterior está nos coeficientes utilizados. O padrão BT.709 foi ajustado para oferecer maior fidelidade visual em monitores e televisores de alta definição, produzindo imagens em tons de cinza mais compatíveis com a percepção humana e com as tecnologias modernas de exibição digital.

2.2.3). Média Aritmética

O método da média aritmética é uma das formas mais simples de converter uma imagem colorida para tons de cinza. Nesse procedimento, considera-se que os três canais de cor (vermelho, verde e azul) possuem a mesma importância na composição do brilho da imagem. Assim, o valor final da luminância é obtido pela média dos valores dos três canais.

Diferentemente dos métodos baseados na percepção visual humana, essa técnica não atribui pesos diferentes para cada cor. Isso significa que o sistema ignora o fato de que o olho humano percebe o verde com maior intensidade e o azul com menor sensibilidade. Como consequência, o resultado pode apresentar diferenças em relação à forma como percebemos o brilho e contraste naturalmente.

A conversão é realizada pela seguinte expressão:

$$Y = (R + G + B) / 3$$

Apesar de sua simplicidade, esse método possui grande importância didática, pois facilita a compreensão inicial do processo de conversão de imagens coloridas para escala de cinza. Além disso, apresenta baixo custo computacional, sendo útil em aplicações simples ou em contextos educacionais introdutórios sobre processamento digital de imagens.

Por simplicidade, a tabela a seguir resume os três métodos:

Método	Fórmula	Padrão	Uso Típico
Luminância	$Y = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B$	ITU-R BT.601	TV analógica, câmeras
Luma	$Y = 0,2126 \cdot R + 0,7152 \cdot G + 0,0722 \cdot B$	ITU-R BT.709	HDTV, displays modernos
Média aritmética	$Y = (R + G + B) / 3$	—	Uso didático

2.3 Binarização por Limiarização Global

A binarização por limiarização global é uma das técnicas mais utilizadas no processamento digital de imagens para separar objetos do fundo da imagem. O princípio desse método consiste em transformar uma imagem em tons de cinza em uma imagem binária, ou seja, composta apenas por dois valores possíveis: preto e branco.

Nesse processo, define-se previamente um valor de limiar (**T**), geralmente pertencente ao intervalo de intensidade de pixels de 0 a 255. Em seguida, cada pixel da imagem é analisado individualmente. Se o valor de intensidade do pixel for maior ou igual ao limiar estabelecido, ele é convertido para branco (255). Caso contrário, o pixel é convertido para preto (0). Dessa forma, regiões consideradas relevantes podem ser destacadas enquanto o restante da imagem é descartado.

A regra de transformação pode ser representada da seguinte maneira:

$$b(i, j) = \begin{cases} 255, & \text{se } y(i, j) \geq T \\ 0, & \text{se } y(i, j) < T \end{cases}$$

Em que:

- $y(i,j)$ → Representa o valor de intensidade do pixel localizado na posição (i,j) da imagem original;
- (T) → Corresponde ao valor de limiar escolhido;
- $b(i,j)$ → Representa o valor do pixel na imagem binarizada;

A escolha adequada do limiar (T) é um fator fundamental para a qualidade da segmentação obtida. Quando a imagem apresenta um histograma bimodal bem definido, isto é, dois grupos distintos de intensidades correspondentes ao objeto e ao fundo. Já um limiar simples costuma produzir resultados satisfatórios.

Entretanto, em imagens com iluminação não uniforme, presença de sombras ou baixo contraste, a definição de um único limiar global pode não ser suficiente para separar corretamente os elementos da imagem. Nesses casos, técnicas mais avançadas, como limiarização adaptativa ou métodos automáticos de determinação de limiar, tendem a produzir resultados mais robustos e precisos.

2.4 Método de Otsu (1979)

O Método de Otsu, proposto por Nobuyuki Otsu em 1979, é uma das técnicas mais conhecidas e utilizadas para a binarização automática de imagens. Seu principal objetivo é determinar, de forma automática, o valor de limiar ideal capaz de separar os pixels pertencentes ao fundo da imagem (**background**) dos pixels pertencentes aos objetos de interesse (**foreground**).

Diferentemente da limiarização global simples, em que o valor do limiar precisa ser escolhido manualmente, o Método de Otsu analisa estatisticamente a distribuição dos níveis de cinza da imagem. Para isso, o algoritmo utiliza o histograma da imagem, buscando identificar o valor de limiar que produz a melhor separação possível entre as duas classes de pixels.

A ideia central do método consiste em maximizar a chamada variância interclasse, isto é, a diferença estatística entre o grupo de pixels do fundo e o grupo de pixels do objeto. Quanto maior essa variância, maior tende a ser a separação entre as duas regiões da imagem, resultando em uma segmentação mais eficiente.

A variância interclasse é dada pela seguinte expressão:

$$\sigma_B^2(T) = w_b(T) \cdot w_f(T) \cdot [\mu_b(T) - \mu_f(T)]^2$$

Em que:

- $\omega_b(T) \rightarrow$ Representa a proporção de pixels classificados como fundo;
- $\omega_f(T) \rightarrow$ Representa a proporção de pixels classificados como objeto;
- $\mu_b(T) \rightarrow$ Corresponde à média das intensidades dos pixels do fundo;
- $\mu_f(T) \rightarrow$ Corresponde à média das intensidades dos pixels do objeto;
- $\sigma_B^2(T) \rightarrow$ Representa a variância interclasse associada ao limiar (T);

O algoritmo avalia todos os possíveis valores de limiar no intervalo de 0 a 255 e seleciona aquele que maximiza a variância interclasse. Esse processo pode ser representado matematicamente por:

$$T^* = \operatorname{argmax}_{T \in \{0,1,\dots,255\}} \sigma_B^2(T)$$

Isso significa que o limiar ótimo (T^*) será aquele que proporciona a maior separação estatística entre fundo e objeto. Além disso, uma das grandes vantagens do Método de Otsu é sua eficiência computacional. A implementação utiliza diretamente o histograma da imagem para calcular os pesos e médias de maneira incremental, evitando recalcular informações repetidamente. Dessa forma, o algoritmo apresenta complexidade computacional **O(N + 256)**, em que (N) representa o número total de pixels da imagem.

Na prática, o Método de Otsu produz excelentes resultados em imagens cujo histograma apresenta dois grupos bem definidos de intensidades, caracterizando distribuições bimodais. Entretanto, em situações com iluminação irregular, sombras ou baixo contraste, o desempenho do método pode ser reduzido, tornando necessária a utilização de técnicas mais avançadas de segmentação adaptativa.

2.5 Limiarização Adaptativa Local

A limiarização adaptativa local é uma técnica de binarização desenvolvida para lidar com imagens que apresentam variações de iluminação, sombras ou diferenças significativas de contraste ao longo da cena. Nessas situações, o uso de um único limiar global pode

produzir resultados inadequados, pois regiões muito claras ou muito escuras podem ser classificadas incorretamente.

Para resolver esse problema, a limiarização adaptativa calcula um limiar específico para cada pixel da imagem, levando em consideração apenas as características de sua vizinhança local. Em vez de utilizar um único valor fixo para toda a imagem, o método ajusta dinamicamente o limiar de acordo com as intensidades presentes em cada região analisada.

O cálculo do limiar local é realizado a partir da média dos pixels presentes em uma janela quadrada centrada no pixel analisado. Em seguida, subtrai-se uma constante de ajuste (C), responsável por controlar a sensibilidade da segmentação. Esse procedimento pode ser descrito pela seguinte expressão:

$$T(i, j) = \text{média}(\text{vizinhança}(i, j)) - C$$

Em que:

- $T(i, j)$ → Representa o limiar calculado para o pixel localizado na posição (i, j) ;
- A vizinhança corresponde aos pixels presentes em uma janela quadrada ao redor do pixel analisado;
- (C) é uma constante de ajuste fino, geralmente pequena, utilizada para melhorar a separação entre objeto e fundo;

Após o cálculo do limiar local, cada pixel é classificado individualmente de acordo com a seguinte regra:

$$b(i, j) = \begin{cases} 255, & \text{se } y(i, j) \geq T(i, j) \\ 0, & \text{se } y(i, j) < T(i, j) \end{cases}$$

Neste caso:

- $y(i, j)$ → Representa a intensidade do pixel original;
- $b(i, j)$ → Corresponde ao valor do pixel na imagem binarizada;

A vizinhança utilizada no cálculo normalmente é definida por uma janela quadrada de dimensões ($W \times W$), centrada no pixel analisado. O tamanho dessa janela influencia diretamente o resultado da segmentação: janelas menores tornam o método mais sensível a detalhes locais e ruídos, enquanto janelas maiores produzem resultados mais suaves e estáveis.

A constante (**C**), frequentemente escolhida com valores próximos de 2, atua como um parâmetro de refinamento. Ajustes nesse valor podem tornar a binarização mais rígida ou mais permissiva, dependendo das características da imagem analisada.

Embora apresente desempenho superior em imagens com iluminação irregular, a limiarização adaptativa possui maior custo computacional em comparação aos métodos globais. Isso ocorre porque o limiar precisa ser recalculado para cada pixel da imagem, utilizando informações de sua vizinhança local. Assim, sua complexidade computacional é da ordem de ($O(N \cdot W^2)$). Daí:

- (**N**) representa o número total de pixels da imagem;
- (W^2) corresponde ao número de pixels presentes na janela de vizinhança;

Apesar do maior custo computacional, essa técnica é amplamente utilizada em aplicações de visão computacional e processamento digital de imagens, especialmente em cenários com iluminação não uniforme, documentos digitalizados, reconhecimento de caracteres e segmentação de objetos em ambientes reais.

3. Implementação

3.1 Arquitetura do Pipeline

O pipeline de processamento segue quatro etapas sequenciais, implementadas como funções independentes e testáveis:

- **Etapa 1** (Carregamento) → leitura do arquivo de imagem e conversão para array NumPy no formato RGB.
- **Etapa 2** (Conversão para tons de cinza) → aplicação do método escolhido (luminância, luma ou média) pixel a pixel via operações vetorizadas.
- **Etapa 3** (Binarização) → aplicação do algoritmo de limiarização selecionado sobre o array de cinza.
- **Etapa 4** (Saída) → salvamento das imagens intermediárias e geração do painel de visualização com histograma.

3.2 Estrutura do Código

O arquivo do código de binarização em python está organizado em cinco módulos lógicos:

```
binarizacao.py
├── 1. CONVERSÃO PARA TONS DE CINZA
│   ├── rgb_para_cinza_luminancia()
│   ├── rgb_para_cinza_media()
│   └── rgb_para_cinza_luma()
├── 2. BINARIZAÇÃO
│   ├── binarizar_limiar_global()
│   ├── binarizar_otsu() → retorna (imagem, limiar_encontrado)
│   └── binarizar_adaptativo()
├── 3. VISUALIZAÇÃO
│   └── visualizar_resultados()
├── 4. PIPELINE PRINCIPAL
│   └── processar_imagem()
└── 5. INTERFACE CLI (argparse)
```

3.3 Funções Principais

3.3.1). rgb_para_cinza_luminancia

A função “*rgb_para_cinza_luminancia*” tem como objetivo converter uma imagem colorida no formato RGB para uma imagem em tons de cinza utilizando o método da luminância. Nesse processo, a conversão não é realizada por uma simples média entre as cores, mas sim por uma combinação ponderada dos canais vermelho, verde e azul, considerando a forma como o sistema visual humano percebe o brilho das diferentes cores.

A função recebe como entrada um array da biblioteca NumPy com dimensões (H, W, 3), no qual:

- **(H)** → Representa a altura da imagem (**Height**);
- **(W)** → Representa a largura da imagem (**Width**);
- O valor **(3)** corresponde aos três canais de cor do modelo RGB: vermelho (**Red**), verde (**Green**) e azul (**Blue**);

Os valores dos pixels são armazenados no tipo “*uint8*”, que representa números inteiros sem sinal no intervalo de 0 a 255. Esse padrão é amplamente utilizado em imagens digitais, pois cada canal de cor pode assumir 256 níveis diferentes de intensidade.

Durante o processamento, a multiplicação dos valores dos pixels pelos coeficientes da equação de luminância é realizada utilizando o tipo “*float64*”. Essa escolha é importante para evitar problemas de **overflow**, que podem ocorrer quando operações matemáticas produzem valores fora do intervalo suportado pelo tipo original “*uint8*”. O uso de precisão em ponto flutuante também garante maior exatidão nos cálculos intermediários.

Após o cálculo da luminância, os valores resultantes passam por uma etapa de **clipping**, em que qualquer valor inferior a 0 é ajustado para 0, e qualquer valor superior a 255 é ajustado para 255. Esse procedimento assegura que todos os pixels permaneçam dentro da faixa válida de intensidades de uma imagem digital em 8 bits.

Por fim, os dados são convertidos novamente para o tipo “*uint8*”, produzindo uma imagem final em tons de cinza com dimensões (H, W). Nesse novo formato, cada pixel passa a possuir apenas um valor de intensidade luminosa, em vez de três canais de cor independentes.

Esse tipo de conversão é amplamente utilizado em aplicações de processamento digital de imagens, visão computacional e aprendizado de máquina, pois reduz a complexidade dos dados sem eliminar as principais informações estruturais presentes na imagem original.

3.3.2). binarizar_otsu

A função “**binarizar_otsu**” realiza a binarização automática de imagens em tons de cinza utilizando o Método de Otsu. Seu principal objetivo é determinar, de maneira automática, o valor de limiar mais adequado para separar os pixels pertencentes ao fundo da imagem dos pixels correspondentes aos objetos de interesse.

O processo inicia-se com o cálculo do histograma da imagem. Para isso, a função percorre todos os pixels da imagem em tons de cinza e contabiliza quantas vezes cada nível de intensidade aparece. Como imagens em escala de cinza de 8 bits possuem valores entre 0 e 255, o histograma resultante contém 256 posições, cada uma representando a frequência de um determinado nível de cinza.

A partir desse histograma, o algoritmo percorre todos os possíveis valores de limiar, de 0 a 255. Para cada valor analisado, são calculadas estatísticas relacionadas às duas classes formadas pela divisão da imagem, isto é:

- os pixels abaixo do limiar, considerados pertencentes ao fundo (**background**);
- e os pixels acima do limiar, considerados pertencentes ao objeto (**foreground**).

Durante essa etapa, a função calcula de maneira incremental a variância interclasse, medida estatística utilizada pelo Método de Otsu para avaliar a qualidade da separação entre as duas regiões da imagem. O limiar que maximiza essa variância é selecionado como o limiar ótimo da segmentação.

A implementação incremental torna o algoritmo mais eficiente, pois evita o recálculo completo das médias e proporções das classes a cada novo valor de limiar testado. Dessa forma, o processamento é realizado de maneira mais rápida, mesmo para imagens com grande quantidade de pixels.

Após determinar automaticamente o limiar ótimo, a função aplica a binarização da imagem. Isto resulta no seguinte processo:

- pixels com intensidade maior ou igual ao limiar são convertidos para branco (255);
- pixels com intensidade inferior ao limiar são convertidos para preto (0).

Como resultado, a função retorna uma tupla² contendo:

1. A imagem binarizada;
2. O valor do limiar calculado automaticamente pelo Método de Otsu;

Esse retorno adicional do limiar é particularmente útil em aplicações de análise e depuração, pois permite que o programador ou pesquisador inspecione o valor selecionado pelo algoritmo e avalie seu comportamento em diferentes tipos de imagem.

Por fim, a função “**binarizar_otsu**” é amplamente aplicável em tarefas de processamento digital de imagens, como segmentação de objetos, reconhecimento de padrões, leitura automática de documentos, visão computacional e pré-processamento para algoritmos de aprendizado de máquina.

3.3.3). binarizar_adaptativo

A função “**binarizar_adaptativo**” realiza a binarização de imagens em tons de cinza utilizando a técnica de limiarização adaptativa local. Diferentemente dos métodos globais, que utilizam um único valor de limiar para toda a imagem, essa abordagem calcula um limiar específico para cada pixel, levando em consideração as características da região ao seu redor.

Esse tipo de técnica é especialmente útil em imagens com iluminação irregular, sombras ou variações locais de contraste, situações em que um limiar global pode produzir resultados insatisfatórios. Ao adaptar o limiar às condições locais da imagem, o algoritmo consegue segmentar objetos de maneira mais precisa.

Para calcular o limiar de cada pixel, a função analisa uma janela de vizinhança ao redor da posição atual. A média das intensidades presentes nessa região é utilizada como referência para determinar se o pixel deve ser convertido para preto ou branco na imagem binarizada.

² Em programação, uma tupla é uma estrutura de dados usada para armazenar múltiplos valores em uma única variável. Ela funciona de maneira semelhante a uma lista, porém com uma diferença importante: seus elementos normalmente são imutáveis, ou seja, não podem ser alterados depois de criados. As tuplas são muito utilizadas quando uma função precisa retornar mais de uma informação relacionada, mantendo os dados organizados e agrupados em uma única estrutura.

Um aspecto importante da implementação é o tratamento das bordas da imagem. Pixels localizados próximos às extremidades não possuem vizinhança completa, o que poderia gerar erros ou artefatos visuais durante o processamento. Para resolver esse problema, a função utiliza o método “*np.pad*” da biblioteca **NumPy** com o parâmetro “*mode='reflect'*”.

Nesse modo, as bordas da imagem são expandidas por reflexão, isto é, os pixels próximos às extremidades são espelhados artificialmente para criar uma continuidade visual ao redor da imagem. Esse procedimento evita descontinuidades abruptas e reduz artefatos que poderiam surgir caso fossem utilizados valores fixos ou preenchimento com zeros.

Após o preenchimento das bordas, o algoritmo percorre explicitamente cada pixel da imagem. Para cada posição:

1. É extraída a janela local de vizinhança;
2. Calcula-se a média dos valores dessa região;
3. Define-se o limiar local;
4. Compara-se o valor do pixel central com esse limiar;
5. O pixel é convertido para branco (255) ou preto (0);

A implementação realiza esses cálculos de forma explícita e iterativa, sem utilizar otimizações internas complexas ou operações vetorizadas ocultas. Essa escolha torna o algoritmo mais didático e transparente, facilitando o entendimento do funcionamento da limiarização adaptativa passo a passo.

Embora essa abordagem seja computacionalmente mais lenta em comparação com versões otimizadas, ela possui grande valor educacional, pois evidencia claramente cada etapa do processamento da imagem. Além disso, permite ao programador compreender com maior profundidade como os limiares locais são calculados e aplicados durante a segmentação.

3.3.4). **processar_imagem**

A função “**processar_imagem**” atua como a etapa central de coordenação do fluxo de processamento digital da imagem. Sua principal responsabilidade é organizar e executar, de maneira integrada, todas as etapas necessárias para a leitura, processamento, binarização, salvamento e visualização dos resultados obtidos.

A função recebe como entrada o caminho da imagem que será processada, além de parâmetros opcionais que permitem ajustar o comportamento dos algoritmos utilizados. Esses parâmetros podem incluir, por exemplo, valores de limiar, tamanho de janela para métodos adaptativos ou diretórios de saída para armazenamento dos resultados.

Após carregar a imagem, a função executa os diferentes métodos de binarização implementados no sistema. Em particular, aplica simultaneamente:

- A binarização por limiarização global;
- A binarização automática pelo Método de Otsu;
- A limiarização adaptativa local;

Essa abordagem permite comparar diretamente o comportamento de diferentes técnicas de segmentação sobre uma mesma imagem, evidenciando as vantagens e limitações de cada método em diferentes condições de iluminação, contraste e distribuição de intensidades.

Depois do processamento, as imagens resultantes são armazenadas em arquivos de saída, possibilitando a análise posterior dos resultados gerados por cada algoritmo. O salvamento automático também facilita a documentação experimental e a reprodução dos testes realizados.

Além disso, a função chama rotinas de visualização responsáveis por exibir as imagens processadas. Essa etapa é importante para permitir uma comparação visual entre:

- A imagem original;
- A imagem em tons de cinza;
- Os diferentes resultados de binarização produzidos pelos algoritmos;

Outro aspecto relevante da função é a geração de um log detalhado de execução. Durante o processamento, são impressas informações estatísticas relacionadas a cada etapa do pipeline, como:

- Dimensões da imagem;
- Tipo dos dados armazenados;
- Valores mínimos e máximos de intensidade;
- Limiares calculados automaticamente;
- Parâmetros utilizados pelos algoritmos;

- Informações sobre os arquivos gerados;

Esse mecanismo de registro possui grande importância tanto para fins de depuração quanto para análise experimental, permitindo acompanhar detalhadamente o comportamento do processamento e identificar possíveis problemas ou inconsistências nos resultados.

Dessa forma, a função “**processar_imagem**” desempenha o papel de orquestradora do sistema, integrando todas as etapas do pipeline de processamento digital de imagens em uma única interface de execução organizada, reutilizável e de fácil manutenção.

4. Análise dos Algoritmos

A tabela a seguir compara a complexidade de tempo e espaço de cada etapa do pipeline, onde $N = W \times H$ é o número total de pixels da imagem e W representa o tamanho da janela no método adaptativo:

Etapa	Tempo	Espaço	Observação
Conversão RGB→Cinza	$O(N)$	$O(N)$	$N = W \times H$ (total de pixels)
Binarização Global	$O(N)$	$O(N)$	Operação vetorizada (NumPy)
Método de Otsu	$O(N + 256)$	$O(N + 256)$	Histograma com 256 bins
Adaptativo Local	$O(N \cdot W^2)$	$O(N)$	W = tamanho da janela

O método adaptativo é o gargalo computacional do pipeline. Para uma imagem Full HD ($1920 \times 1080 = 2.073.600$ pixels) com janela 11×11 , o número de operações é da ordem de 2×10^8 , o que pode levar alguns segundos em Python puro. Para aplicações que demandem maior desempenho, recomenda-se substituir o loop explícito por convolução via *scipy.ndimage.uniform_filter*, reduzindo a complexidade prática para $O(N)$.

Os métodos global e de Otsu são eficientes em todas as situações, sendo adequados para processamento em lote de grandes volumes de imagens.

5. Resultados e Discussão

5.1 Conversão para Tons de Cinza

As três fórmulas produzem resultados visualmente distintos, especialmente em imagens com cores saturadas. A Luminância (**BT.601**) preserva melhor a percepção de contraste do sistema visual humano: regiões verdes tendem a parecer mais claras, enquanto azuis e vermelhos têm menor contribuição. A Média Aritmética frequentemente gera imagens com contraste reduzido em cenas coloridas, pois ignora a sensibilidade diferencial do olho humano às frequências de cor.

Em imagens com tons de pele, a diferença entre **BT.601** e **BT.709** é marginal. Para imagens médicas ou científicas onde a colorimetria é calibrada para monitores modernos, o padrão **BT.709** é o mais indicado.

5.2 Comparação dos Métodos de Binarização

O Limiar Global com $T = 128$ é adequado para imagens com histograma bimodal simétrico, mas falha em imagens escuras ou muito claras. O Método de Otsu supera o limiar fixo em praticamente todos os casos, determinando automaticamente o ponto de separação ideal entre fundo e objeto. Em testes com imagens de documentos (texto preto sobre fundo branco), o Otsu consistentemente encontrou limiares entre 120 e 180, produzindo segmentações limpas.

A Limiarização Adaptativa se destaca em imagens com gradiente de iluminação, como fotografias de documentos capturadas com iluminação lateral. Nesses casos, o Otsu tende a "perder" texto em regiões mais escuras, enquanto o adaptativo mantém a legibilidade em toda a extensão da imagem. O parâmetro **C** recomendado é 2 (padrão), podendo ser aumentado para imagens com muito ruído.

5.3 Impacto do Ruído

Ruído gaussiano adicionado à imagem afeta diferentemente os três métodos de binarização. O limiar global e o Otsu são robustos a ruídos de baixa intensidade, pois operam sobre a estatística global da imagem. O método adaptativo, por ser sensível à vizinhança local, pode amplificar ruídos em regiões homogêneas. Recomenda-se aplicar um filtro de suavização (Gaussiano ou mediana) antes da binarização adaptativa em imagens ruidosas.

6. Conclusão

Este projeto demonstrou a implementação completa de um pipeline de binarização de imagens digitais em Python, abrangendo desde a teoria matemática até o código executável. A implementação explícita dos algoritmos, sem recorrer a funções de alto nível, torna o projeto especialmente adequado para fins didáticos em disciplinas de estudos sobre Machine Learning.

Dos três métodos de binarização implementados, o Método de Otsu mostrou-se o mais robusto para uso geral, determinando automaticamente o limiar ótimo sem intervenção

do usuário. A limiarização adaptativa é indispensável em cenários com iluminação não uniforme, especialmente para digitalização de documentos. O limiar global simples, embora limitado em casos práticos, é valioso didaticamente por sua simplicidade e baixo custo computacional.

7. Referências Bibliográficas

GONZALEZ, R. C.; WOODS, R. E. Processamento Digital de Imagens. 3. ed. São Paulo: Pearson Prentice Hall, 2010.

OTSU, N. A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, v. 9, n. 1, p. 62–66, 1979.

ITU-R BT.601. Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide Screen 16:9 Aspect Ratios. International Telecommunication Union, 2011.

ITU-R BT.709. Parameter Values for the HDTV Standards for Production and International Programme Exchange. International Telecommunication Union, 2015.

NIBLACK, W. An Introduction to Digital Image Processing. Englewood Cliffs: Prentice Hall, 1986.

SAUVOLA, J.; PIETIKÄINEN, M. Adaptive Document Image Binarization. Pattern Recognition, v. 33, n. 2, p. 225–236, 2000.

VAN DER WALT, S. et al. scikit-image: Image Processing in Python. PeerJ, v. 2, e453, 2014.